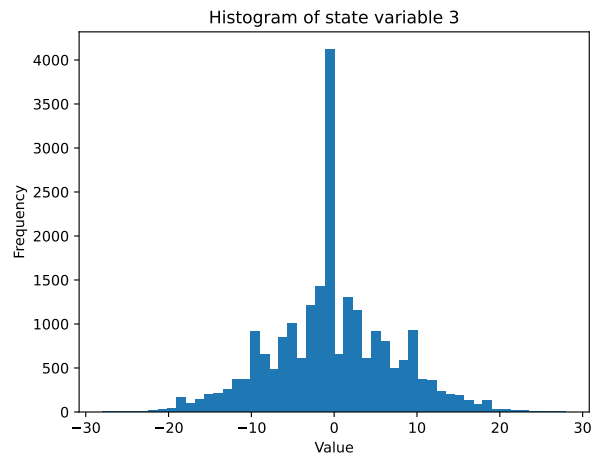
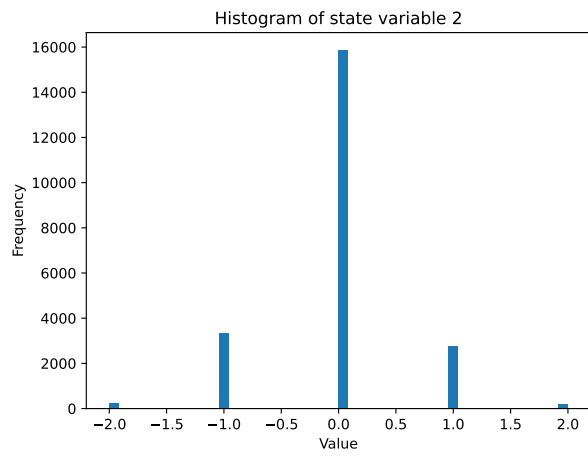
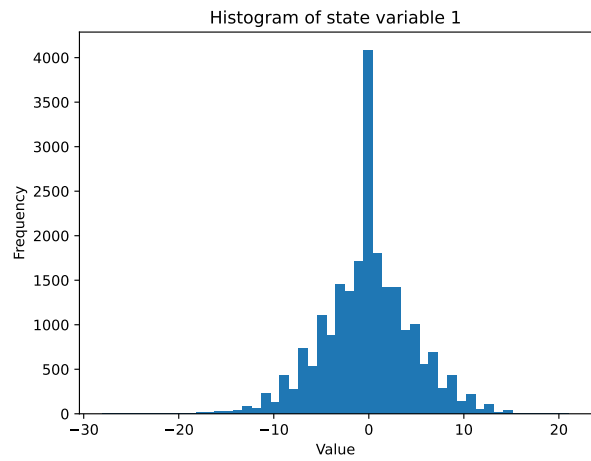
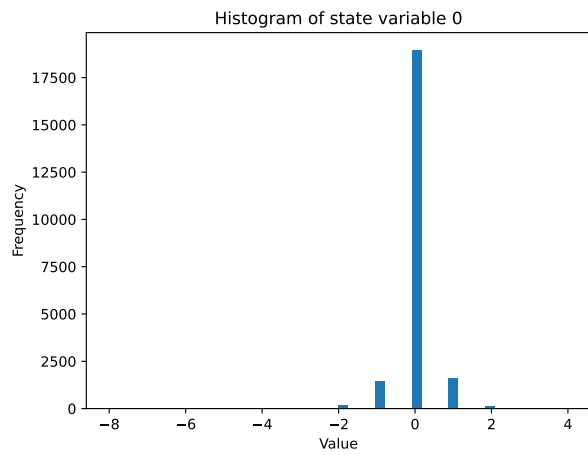


STA 5635 HW 12

Palmer Swanson

April 9, 2026

Part 1



State Variable	Min	Max	Num Discrete Values
0	-8	4	13
1	-28	21	50
2	-2	2	5
3	-28	28	57
Total			185250

Parts 2, 3, and 4

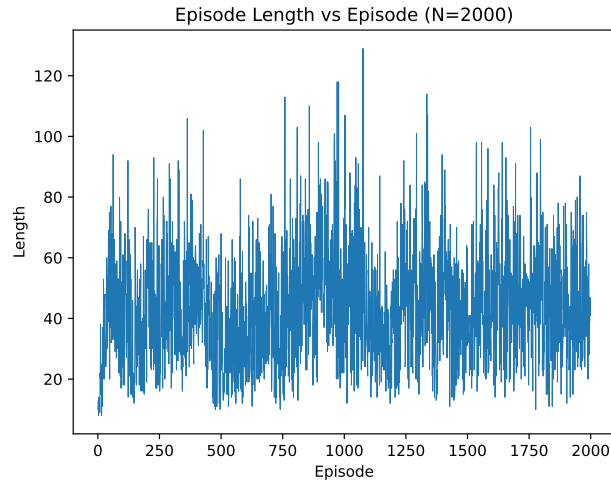


Figure 2: Episode Length vs Episode (N=2000)

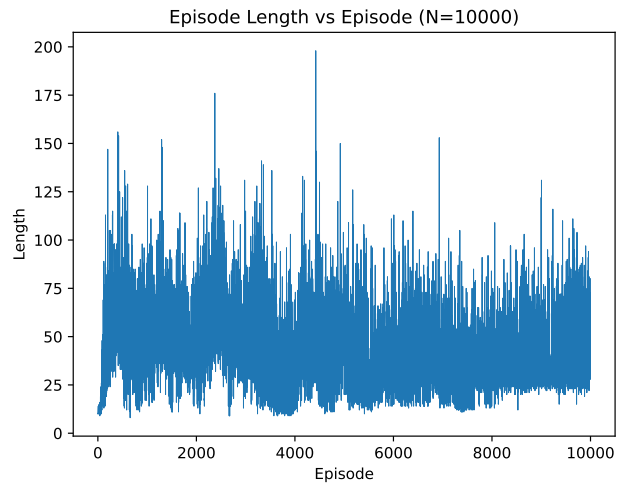


Figure 3: Episode Length vs Episode (N=10000)

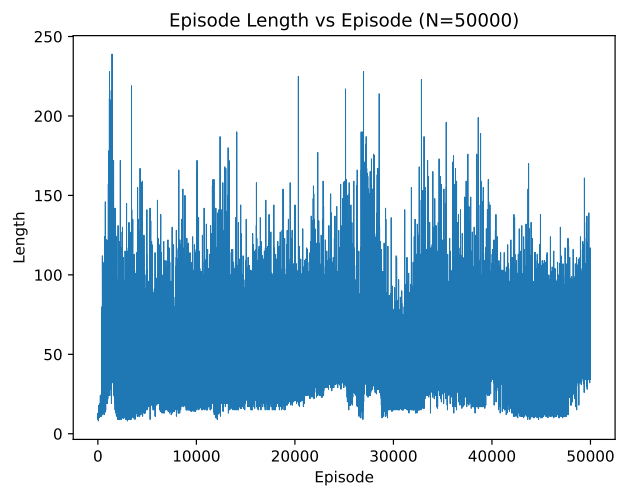


Figure 4: Episode Length vs Episode (N=50000)

N	% Nonzero (Random)	% Nonzero (Max)	Avg Ep. Length (Last 1000)
2000	1.203	1.702	44.322
10000	2.508	3.404	41.164
50000	4.719	6.106	63.964

Code

```
1 import gymnasium as gym
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import random
6
7 #https://gymnasium.farama.org/introduction/basic_usage/
8 #I think the first three states are position, velocity, and angle.
9 #not sure what the fourth state is
10
11 #https://gymnasium.farama.org/environments/classic_control/cart_pole/
12 #ah. fourth is angular velocity.
13 #need to create a cartpole environment
14 env = gym.make("CartPole-v1")
15
16 n_episodes = 1000
17
18 states = []
19
20 #part a
21 #loop over and collect the states of 1000 episodes
22 for i in range(n_episodes):
23
24     #reset enviro at the start of each episode
25     state, info = env.reset()
26     done = False
27
28     #run this until done
29     while not done:
30         #append to the states list
31         states.append(state)
32
33         #take a random action (0 or 1) and observe the next state and reward
34         a = random.randrange(2)
35         observation, reward, terminated, truncated, info = env.step(a)
36
37         #episode ends if one of these is true
38         done = terminated or truncated
39
40         #update state
41         state = observation
42
43 env.close()
44
45 #need to multiply by 10 and converting to integers
```

```

46 states = (np.array(states)*10).astype(int)
47
48 #plot histograms and save
49 #using 50 bins for each state variable, and save as pdfs in the figures folder
50 for i in range(4):
51     plt.figure()
52     plt.hist(states[:, i], bins=50)
53     plt.title(f"Histogram of state variable {i}")
54     plt.xlabel("Value")
55     plt.ylabel("Frequency")
56     plt.savefig(f"assignments/hw12/figures/hist_state_variable_{i}.pdf",
57                 dpi=400, bbox_inches="tight")
58     plt.close()
59     #plt.show()
60
61 #need to find the range of each of the state variables
62 #also need to report the number of discrete state values for each state variable
63 mins = states.min(axis=0)
64 maxs = states.max(axis=0)
65 num_vals = maxs - mins + 1
66
67 #get total number of discrete states
68 num_total_states = np.prod(num_vals)
69 #print(f"Total number of discrete states: {num_total_states}")
70
71 #build dataframe
72 df = pd.DataFrame({
73     "State_Variable": [0, 1, 2, 3],
74     "Min": mins,
75     "Max": maxs,
76     "Num_Discrete_Values": num_vals
77 })
78
79 #add a total row
80 total_row = pd.DataFrame({
81     "State_Variable": ["Total"],
82     "Min": [""],
83     "Max": [""],
84     "Num_Discrete_Values": [num_total_states]
85 })
86
87 df = pd.concat([df, total_row], ignore_index=True)
88
89 #export table as latex
90 latex_table = df.to_latex(index=False)
91
92 with open("assignments/hw12/output/state_summary_part1.tex", "w") as f:

```

```

92     f.write(latex_table)
93
94
95
96 #parts 2, 3, and 4
97 #loop over these
98 N_values = [2000, 10000, 50000]
99 gamma = 0.9
100
101 results = []
102
103 for N in N_values:
104     print(f"\nN={N}")
105
106     #initialize Q table
107     Q = np.zeros((num_vals[0], num_vals[1], num_vals[2], num_vals[3], 2))
108
109     for i in range(N):
110         state, info = env.reset()
111         done = False
112
113         while not done:
114             #convert state to discrete state and clip to the min and max values
115             discrete_state = (state*10).astype(int)
116             discrete_state = np.clip(discrete_state, mins, maxs)
117             s_idx = discrete_state - mins
118
119             #take random action
120             a = random.randrange(2)
121             next_state, reward, terminated, truncated, info = env.step(a)
122             done = terminated or truncated
123
124             #convert next state to discrete state and clip to the min and max
125             values
126             disc_next = (next_state * 10).astype(int)
127             disc_next = np.clip(disc_next, mins, maxs)
128             s_next_idx = disc_next - mins
129
130             #update Q table
131             Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3], a] = (
132                 reward + gamma * np.max(
133                     Q[s_next_idx[0], s_next_idx[1], s_next_idx[2], s_next_idx[3]]
134                 )
135             )
136
137             state = next_state

```

```

138     #percent nonzero after random phase
139     percent1 = 100 * np.count_nonzero(Q) / Q.size
140
141     #ready to store episode lengths (last 1000)
142     eps = []
143
144     for i in range(N):
145         state, info = env.reset()
146         done = False
147         steps = 0
148
149         #stop any episode with more than 2000 steps
150         while not done and steps < 2000:
151             #convert state to discrete state
152             discrete_state = (state*10).astype(int)
153             discrete_state = np.clip(discrete_state, mins, maxs)
154             s_idx = discrete_state - mins
155
156             a = np.argmax(Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3]])
157
158             next_state, reward, terminated, truncated, info = env.step(a)
159             done = terminated or truncated
160
161             #convert next state to discrete state
162             disc_next = (next_state * 10).astype(int)
163             disc_next = np.clip(disc_next, mins, maxs)
164             s_next_idx = disc_next - mins
165
166             #update Q table. Basically the same as before but we use max
167             Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3], a] = (
168                 reward + gamma * np.max(
169                     Q[s_next_idx[0], s_next_idx[1], s_next_idx[2], s_next_idx[3]]
170                 )
171             )
172
173             #set state to next state and increment steps
174             state = next_state
175             steps += 1
176
177             #memorize episode length
178             eps.append(steps)
179
180     #percent non zero after max action phase
181     percent2 = 100 * np.count_nonzero(Q) / Q.size
182     avg_last_1000 = np.mean(eps[-1000:])
183
184     #save objects we need to report in a table

```



```

185     results.append({
186         "N": N,
187         "%_Nonzero_(Random)": percent1,
188         "%_Nonzero_(Max)": percent2,
189         "Avg_Ep._Length_(Last_1000)": avg_last_1000
190     })
191
192     #save plot, thin the linewidth (I thought it looked better)
193     plt.figure()
194     plt.plot(eps, linewidth=0.3)
195     plt.title(f"Episode_Length_vs_Episode_(N={N})")
196     plt.xlabel("Episode")
197     plt.ylabel("Length")
198     plt.savefig(f"assignments/hw12/figures/episode_length_N{N}.pdf", dpi=300,
199               bbox_inches="tight")
200     plt.close()
201
202     #convert results to df and export
203     results_df = pd.DataFrame(results)
204
205     #need escape for the % signs
206     latex_table = results_df.to_latex(
207         index=False,
208         escape=True,
209         float_format="%.3f"
210     )
211
212     #save latex table
213     with open("assignments/hw12/output/results_summary.tex", "w") as f:
214         f.write(latex_table)

```